

Bàn về “0” và “1” trong thi Tin học

Wu Sen

Trường Trung học số 2 Thạch Gia Trang, Hà Bắc

2009

Nguồn gốc: Bàn về “0” và “1” trong thi Tin học

IOI China candidate team papers / 2009 / Wu Sen

Abstract

Máy tính hiện đại biểu diễn và xử lý thông tin bằng hệ nhị phân. Không chỉ trong kiến trúc máy tính, tư tưởng nhị phân còn xuất hiện rất nhiều trong thi Tin học: từ cấu trúc dữ liệu, nén trạng thái, mô hình hóa bằng bit cho tới cây nhị phân 0/1. Bài viết dùng bốn ví dụ để tổng kết cách đưa hai ký hiệu 0 và 1 vào phân tích thuật toán.

Lời nói đầu

Trong thư viện cung điện Gotha ở Thuringia, Đức, từng lưu giữ một bản thảo quý có nhan đề đại ý là: “1 và 0, nguồn gốc kỳ diệu của mọi con số”. Cách mở đầu ấy cho thấy từ rất sớm người ta đã nhìn thấy sức biểu đạt đặc biệt của hai ký hiệu đơn giản này.

Ngày nay, hệ nhị phân là hệ đếm được dùng rộng rãi trong kỹ thuật tính toán. Máy tính điện tử hiện đại đều dùng hai ký hiệu 0 và 1, vì chúng đơn giản, thuận tiện và dễ hiện thực bằng mạch điện. Mọi thông tin bên trong máy tính, từ số, chữ cái, ký hiệu, chữ Hán, âm thanh cho tới hình ảnh, cuối cùng đều được mã hóa dưới dạng nhị phân. Quy tắc tiến vị của hệ nhị phân là “đủ 2 thì nhớ 1”; chữ số 1 ở những vị trí khác nhau biểu thị những giá trị tăng gấp đôi từ phải sang trái. Ngoài biểu diễn số, các chuẩn mã như ASCII cũng là ví dụ trực tiếp của việc đưa thông tin về dãy bit.

Bài viết không bàn về kiến trúc máy tính, mà dùng một số bài toán OI để chỉ ra rằng tư tưởng nhị phân còn là một công cụ thiết kế thuật toán: dùng bit để mã hóa trạng thái, dùng biểu diễn nhị phân để tổ chức dữ liệu, và dùng cây nhị phân 0/1 để tối ưu các phép toán bit.

Từ khóa. Nhị phân; thập phân; cây Fenwick; nén trạng thái; cây nhị phân 0/1.

Contents

1	Ứng dụng trong cấu trúc dữ liệu	2
1.1	Ví dụ 1: Matrix	2
1.1.1	Từ một chiều tới hai chiều	2
1.1.2	Cây Fenwick và tư tưởng nhị phân	3

2	Ứng dụng trong tư duy giải bài	3
2.1	Ví dụ 2: Sudoku	3
2.2	Ví dụ 3: Requirements	4
2.3	Ví dụ 4: Cow XOR	5
3	Tổng kết	5
4	Tài liệu tham khảo	5
5	Phụ lục: nguyên đề các ví dụ	5
5.1	Matrix	5
5.2	Sudoku	6
5.3	Requirements	7
5.4	Cow XOR	7

1 Ứng dụng trong cấu trúc dữ liệu

1.1 Ví dụ 1: Matrix

Cho ma trận $M \times N$, mỗi ô chỉ có giá trị 0 hoặc 1. Ban đầu mọi ô đều bằng 0. Mỗi thao tác cập nhật chọn một hình chữ nhật có góc trái trên (x_1, y_1) và góc phải dưới (x_2, y_2) , rồi đảo bit mọi ô trong hình chữ nhật. Mỗi truy vấn hỏi giá trị ô (x, y) .

Vì mỗi ô chỉ là 0 hoặc 1, ta chỉ cần biết ô đó bị đảo bao nhiêu lần; đáp án là số lần đảo modulo 2.

1.1.1 Từ một chiều tới hai chiều

Trước hết xét mảng một chiều d độ dài N . Một cập nhật đoạn $[x, y]$ có thể được đổi thành hai điểm mốc:

$$d_x += 1, \quad d_{y+1} += 1.$$

Khi hỏi vị trí x , tính

$$\text{Sum}_x = \sum_{i=1}^x d_i,$$

rồi trả $\text{Sum}_x \bmod 2$.

Với hai chiều, một hình chữ nhật được đánh dấu bằng bốn góc:

$$(x_1, y_1), \quad (x_2 + 1, y_1), \quad (x_1, y_2 + 1), \quad (x_2 + 1, y_2 + 1).$$

Khi truy vấn (x, y) , tính tổng tiền tố hai chiều

$$\text{Sum}_{x,y} = \sum_{i=1}^x \sum_{j=1}^y d_{i,j},$$

và lấy modulo 2.

Để thấy bốn điểm mốc này đúng, chia mặt phẳng bởi hai đường $x = x_1$, $x = x_2 + 1$ và hai đường $y = y_1$, $y = y_2 + 1$, ta được chín vùng. Với một điểm truy vấn (a, b) :

- nếu (a, b) nằm hoàn toàn trước hình chữ nhật, hoàn toàn bên trái, hoặc ở các vùng không vượt qua cả hai biên cần thiết, tổng tiền tố không nhìn thấy điểm mốc nào, nên không đổi;

- nếu (a, b) nằm trong hình chữ nhật cần đảo, tổng tiền tố chỉ nhìn thấy điểm mốc (x_1, y_1) , nên tăng đúng 1;
- nếu (a, b) đã vượt qua một chiều của hình chữ nhật nhưng chưa vượt qua chiều còn lại, tổng tiền tố nhìn thấy đúng hai điểm mốc, nên tăng 2 và không đổi sau khi lấy modulo 2;
- nếu (a, b) nằm sau cả hai chiều, tổng tiền tố nhìn thấy cả bốn điểm mốc, nên tăng 4, cũng không đổi modulo 2.

Vì chỉ vùng bên trong hình chữ nhật tăng lẻ lần, phép đánh dấu bằng bốn góc chính xác tương đương với đảo bit toàn bộ hình chữ nhật.

Lập luận này mở rộng tự nhiên: trong ba chiều cần 8 điểm mốc, còn trong n chiều cần 2^n điểm quanh khối cần cập nhật.

1.1.2 Cây Fenwick và tư tưởng nhị phân

Để duy trì tổng tiền tố, một cấu trúc ngắn gọn và dễ mở rộng là cây Fenwick (Binary Indexed Tree). Mỗi chỉ số được nhìn dưới dạng nhị phân. Nếu

$$\text{lowbit}(x) = x \& (-x),$$

thì nút x lưu tổng của đoạn độ dài $\text{lowbit}(x)$ kết thúc tại x . Ví dụ, $11 = (1011)_2$, và phần bit thấp nhất quyết định kích thước đoạn mà nút này đại diện.

Cập nhật.

```
while x <= max:
    C[x] += delta
    x += x & -x
```

Truy vấn tiền tố.

```
sum = 0
while x > 0:
    sum += C[x]
    x -= x & -x
```

Trong truy vấn, thao tác $x - \text{lowbit}(x)$ xóa bit 1 thấp nhất của x . Một số không vượt quá n có nhiều nhất $\log n$ bit 1, nên truy vấn mất $\mathcal{O}(\log n)$. Cập nhật đi qua các tổ tiên tương ứng và cũng mất $\mathcal{O}(\log n)$. Ưu điểm của Fenwick là mã ngắn, khó sai, có thể mở rộng lên hai hoặc ba chiều, và rất phù hợp với các bài mà thao tác không quá phức tạp.

Tóm lại, đây là một ví dụ điển hình của tư tưởng nhị phân trong cấu trúc dữ liệu: quan hệ giữa số thập phân và biểu diễn nhị phân quyết định trực tiếp khoảng thông tin mà mỗi nút lưu trữ. Nhờ vậy, một bài toán hiệu phân/tổng tiền tố có thể được viết rất ngắn nhưng vẫn đạt độ phức tạp tốt.

2 Ứng dụng trong tư duy giải bài

2.1 Ví dụ 2: Sudoku

Sudoku là bảng 9×9 , chia thành chín khối 3×3 . Cần điền các số 1 đến 9 sao cho mỗi hàng, mỗi cột và mỗi khối đều chứa đủ chín số đúng một lần.

Cách tìm kiếm trực tiếp theo thứ tự ô thường quá chậm. Số bảng hoàn chỉnh của Sudoku rất lớn, khoảng 6.67×10^{21} , nên chỉ dựa vào vết cạn giản là không thực tế.

Một tối ưu quen thuộc là với mỗi ô trống, ghi lại tập chữ số còn có thể đặt, điền trước các ô chỉ còn một khả năng, chọn ô có ít lựa chọn nhất để rẽ nhánh, rồi cập nhật các ô cùng hàng, cột và khối. Tuy nhiên nếu cài bằng tập hợp thông thường, mỗi lần đặt một số phải sửa nhiều danh sách ứng viên và sắp lại thứ tự tìm kiếm; mã dễ dài, dễ sai, và vẫn có thể quá chậm nếu cập nhật không gọn.

Tư tưởng nhị phân biến tập ứng viên của một ô thành một số 9 bit. Bit thứ i biểu thị chữ số i có bị cấm hay không; chẳng hạn 0 nghĩa là có thể đặt, 1 nghĩa là không thể đặt. Mỗi ô khi đó được biểu diễn bởi một số từ 0 tới 511. Trạng thái hàng, cột và khối cũng có thể là mặt nạ bit:

$$\text{available}(r, c) = \neg(\text{row}_r \vee \text{col}_c \vee \text{block}_{r,c}) \& 511.$$

Khi tìm kiếm, ta duyệt các bit 1 còn đặt được bằng các phép bit. Chỉ riêng nén trạng thái bằng bit đã đủ làm bài toán nhanh hơn nhiều; kết hợp thêm chọn ô có ít ứng viên nhất sẽ cho lời giải rất ổn định.

2.2 Ví dụ 3: Requirements

Cho N điểm năm chiều

$$A = (x_1, x_2, x_3, x_4, x_5),$$

cần tìm hai điểm X, Y có khoảng cách Manhattan lớn nhất:

$$\sum_{i=1}^5 |x_i - y_i|.$$

Vì N có thể tới 100000, không thể xét mọi cặp điểm.

Trước hết xét hai chiều. Với hai điểm, đặt $a = x_i - y_i, b = x_j - y_j$. Ta có

$$|a| + |b| = \max\{a + b, a - b, -a + b, -a - b\}.$$

Nói cách khác, ta chỉ cần liệt kê dấu cộng/trừ trước từng tọa độ. Mỗi cách chọn dấu có thể mã hóa bằng một mặt nạ nhị phân: bit 1 nghĩa là dấu cộng, bit 0 nghĩa là dấu trừ. Với mỗi mặt nạ m , tính giá trị tuyến tính

$$F_m(P) = \sum_{i=1}^d s_i x_i, \quad s_i \in \{-1, 1\},$$

rồi lưu $\max F_m(P)$ và $\min F_m(P)$ trên mọi điểm. Đáp án là

$$\max_m (\max_P F_m(P) - \min_P F_m(P)).$$

Với $d = 5$, chỉ có $2^5 = 32$ mặt nạ, nên thuật toán chạy trong $\mathcal{O}(2^d N)$, rất nhỏ so với $\mathcal{O}(N^2)$. Điều kiện quan trọng là số chiều nhỏ hơn nhiều so với số điểm.

Tính đúng của phép biến đổi đến từ đẳng thức hai chiều sau:

$$|a| + |b| = \max\{a + b, a - b, -a + b, -a - b\}.$$

Nếu $ab = 0$, giả sử $a = 0$, về phải chọn được b hoặc $-b$, tức $|b|$. Nếu $ab > 0$, hai số cùng dấu; với $a, b > 0$, biểu thức $a + b$ lớn nhất, còn với $a, b < 0$, biểu thức $-a - b$ lớn nhất. Nếu $ab < 0$, hai số trái dấu; một trong hai biểu thức $a - b$ hoặc $-a + b$ đúng bằng $|a| + |b|$. Vì vậy liệt kê mọi tổ hợp dấu luôn thu được khoảng cách Manhattan.

Với nhiều chiều, lập luận tương tự: mỗi trị tuyệt đối chọn một dấu tương ứng với chiều đó. Mặt nạ nhị phân chỉ là cách mã hóa gọn 2^d khả năng dấu. Để tìm cặp điểm xa nhất, với cùng một mặt nạ ta lấy điểm có giá trị tuyến tính lớn nhất và điểm có giá trị nhỏ nhất; hiệu của chúng chính là ứng viên cho khoảng cách theo mặt nạ ấy.

2.3 Ví dụ 4: Cow XOR

Cho dãy $a_1, a_2, \dots, a_N$, cần chọn một đoạn liên tiếp sao cho xor của đoạn là lớn nhất. Nếu có nhiều đoạn, chọn đoạn có chỉ số kết thúc nhỏ nhất; nếu vẫn hòa, chọn đoạn ngắn nhất. Đặt

$$S_k = a_1 \text{ xor } a_2 \text{ xor } \dots \text{ xor } a_k.$$

Khi đó

$$a_i \text{ xor } a_{i+1} \text{ xor } \dots \text{ xor } a_j = S_j \text{ xor } S_{i-1}.$$

Vì các số nằm trong khoảng 0 tới $2^{21} - 1$, ta có thể lưu các tiền tố S_i trong cây nhị phân 0/1 độ cao 21. Mỗi cạnh ứng với một bit của số; lá lưu chỉ số tiền tố.

Khi chèn

$$S_i = (x_1 x_2 \dots x_{21})_2,$$

ta đi từ gốc xuống lá, tại tầng thứ k đi theo cạnh mang bit x_k . Nếu chưa có nút tương ứng thì tạo nút mới; ở lá lưu chỉ số tiền tố i . Khi cần xử lý hòa, lá nên giữ chỉ số phù hợp với quy tắc của đề: ưu tiên đoạn có kết thúc sớm hơn, rồi đoạn ngắn hơn.

Khi đang xét S_i , để tìm tiền tố S_j làm $S_i \text{ xor } S_j$ lớn nhất, ta đi từ bit cao xuống thấp. Nếu bit hiện tại của S_i là 0, ưu tiên nhánh 1; nếu là 1, ưu tiên nhánh 0. Tức là luôn cố làm bit cao nhất còn lại của xor bằng 1. Chiến lược tham lam đúng vì một bit cao 2^k lớn hơn tổng mọi bit thấp hơn:

$$2^k > \sum_{i=0}^{k-1} 2^i.$$

Mỗi lần truy vấn và chèn một tiền tố mất $\mathcal{O}(21)$, nên toàn bộ thuật toán là $\mathcal{O}(N \log A)$, với $A < 2^{21}$.

3 Tổng kết

Các ví dụ trên cho thấy tư tưởng nhị phân không chỉ là cách biểu diễn số. Trong cấu trúc dữ liệu, nó tạo ra Fenwick tree với mã ngắn và hiệu quả tốt. Trong giải bài, nó giúp nén trạng thái, liệt kê tổ hợp đầu, xây mô hình bằng mặt nạ bit và dùng cây nhị phân 0/1 để tối ưu xor. Khi biết chuyển bài toán từ hệ thập phân quen thuộc sang góc nhìn nhị phân, nhiều lời giải trở nên ngắn hơn, rõ hơn và nhanh hơn.

4 Tài liệu tham khảo

1. Liu Rujia và Huang Liang, *Nghệ thuật thuật toán và thi Tin học*, Nhà xuất bản Đại học Thanh Hoa, ấn bản tháng 1 năm 2004.
2. Peking University Judge Online, <http://acm.pku.edu.cn/>.
3. Zhejiang University Judge Online, <http://acm.zju.edu.cn/>.
4. USA Computing Olympiad, <http://ace.delos.com/>.

5 Phụ lục: nguyên đề các ví dụ

5.1 Matrix

Cho ma trận $N \times N$ gồm các phần tử 0 hoặc 1, ban đầu đều là 0. Có hai loại lệnh:

- C x1 y1 x2 y2: đảo bit mọi ô trong hình chữ nhật có góc trái trên (x_1, y_1) và góc phải dưới (x_2, y_2) .
- Q x y: hỏi giá trị $A[x, y]$.

Có nhiều bộ test, $N \leq 1000$, số lệnh $T \leq 50000$.

Ví dụ.

Input

1

2 10

C 2 1 2 2

Q 2 2

C 2 1 2 1

Q 1 1

C 1 1 2 1

C 1 2 1 2

C 1 1 2 2

Q 1 1

C 1 1 2 1

Q 2 1

Output

1

0

0

1

5.2 Sudoku

Mỗi test là một dòng 81 ký tự mô tả bảng Sudoku theo thứ tự từng hàng. Ký tự là chữ số 1 đến 9 hoặc dấu chấm cho ô trống. Mỗi bảng có đúng một nghiệm; input kết thúc bằng dòng end. Với mỗi test, in bảng đã hoàn thành trên một dòng.

Một ví dụ nguồn được viết thành một dòng dài; khi xuống dòng để đọc dễ hơn, bảng ban đầu có dạng:

```
. 2 7 3 8 . . 1 .
. 1 . . . 6 7 3 5
. . . . . . 2 9
3 . 5 6 9 2 . 8 .
. . . . . . . .
. 6 . 1 7 4 5 . 3
6 4 . . . . . .
9 5 1 8 . . . 7 .
. 8 . . 6 5 3 4 .
```

Đầu ra tương ứng là dòng 81 chữ số của bảng đã điền xong.

5.3 Requirements

Một sinh viên muốn so sánh các yêu cầu tốt nghiệp của nhiều trường đại học. Mỗi trường đều có năm loại yêu cầu: yêu cầu chung, viết, khoa học, ngoại ngữ và chuyên ngành. Mỗi yêu cầu được lượng hóa bằng số lớp cần học, có nhân thêm hệ số phản ánh độ khó tương ứng. Vì vậy mỗi trường đại học được mô tả bằng năm số thực không âm. Độ khác nhau giữa hai trường X, Y là

$$|x_1 - y_1| + |x_2 - y_2| + |x_3 - y_3| + |x_4 - y_4| + |x_5 - y_5|.$$

Với $1 \leq N \leq 100000$, hãy in độ khác nhau lớn nhất giữa hai trường, làm tròn đúng hai chữ số thập phân. Input kết thúc bằng $N = 0$.

Ví dụ.

Input

```
3
2 5 6 2 1.5
1.2 3 2 5 4
7 5 3 2 5
0
```

Output

```
12.80
```

5.4 Cow XOR

Cho N con bò theo thứ tự thứ bậc xã hội, mỗi con có một số trong khoảng 0 tới $2^{21} - 1$. Chọn một đoạn liên tiếp sao cho xor các số trên đoạn là lớn nhất. Nếu có nhiều đoạn, chọn đoạn có điểm kết thúc nhỏ nhất; nếu vẫn hòa, chọn đoạn ngắn nhất. In giá trị xor lớn nhất, vị trí bắt đầu và vị trí kết thúc.

Định dạng vào gồm N ở dòng đầu, sau đó N dòng, mỗi dòng là số được gán cho một con bò theo thứ tự thứ bậc. Đầu ra gồm ba số: giá trị xor lớn nhất, vị trí bắt đầu và vị trí kết thúc của đoạn được chọn.

Ví dụ.

Input

```
5
1
0
5
4
2
```

Output

```
6 4 5
```